

Autoescalamento Horizontal de Aplicações Kubernetes: uma Análise Experimental em Minikube e AWS EKS

Joaquin Barreyro¹

¹Escola Politécnica – Universidade de São Paulo (USP)
São Paulo – SP – Brasil

joaquin.barreyro@alumni.usp.br

Abstract. *This work investigates the use of the Kubernetes Horizontal Pod Autoscaler (HPA) for automatically scaling a web application according to its resource utilization. Two environments are considered: a local Kubernetes cluster deployed with Minikube and a managed cluster deployed on Amazon Web Services using Amazon EKS. The same application and autoscaling strategy are deployed in both environments and evaluated through controlled load tests. The experiments analyze the HPA response, resource consumption, scalability behavior, management complexity, and approximate costs, allowing a comparison between local and cloud-based Kubernetes environments.*

Resumo. *Este trabalho investiga o uso do Horizontal Pod Autoscaler (HPA) do Kubernetes para o escalamento automático de uma aplicação web de acordo com a utilização de recursos. São considerados dois ambientes: um cluster Kubernetes local implantado com Minikube e um cluster gerenciado na Amazon Web Services utilizando Amazon EKS. A mesma aplicação e estratégia de autoescalamento são implantadas nos dois ambientes e avaliadas por meio de testes controlados de carga. Os experimentos analisam a resposta do HPA, o consumo de recursos, o comportamento do escalamento, a complexidade de gerenciamento e os custos aproximados, permitindo a comparação entre os ambientes Kubernetes local e em nuvem.*

1. Introdução

A computação em nuvem permite disponibilizar recursos computacionais de forma flexível, possibilitando que aplicações adaptem sua capacidade de processamento às variações de demanda. Nesse contexto, mecanismos de autoescalamento permitem ajustar os recursos utilizados de acordo com a carga de trabalho.

O Kubernetes fornece recursos para implantação e gerenciamento de aplicações containerizadas por meio de objetos como Pods e Deployments, permitindo controlar o número de réplicas de uma aplicação [Kubernetes 2026a]. Entre seus mecanismos de escalabilidade, o *Horizontal Pod Autoscaler* (HPA) ajusta automaticamente o número de réplicas de acordo com métricas observadas, como a utilização de CPU [Kubernetes 2026b].

O HPA pode ser utilizado tanto em clusters locais quanto em ambientes Kubernetes gerenciados. No Amazon Elastic Kubernetes Service (EKS), o mecanismo de HPA permanece um recurso padrão do Kubernetes, sendo necessária uma fonte de métricas, como o Kubernetes Metrics Server, para fornecer as informações utilizadas no processo de escalamento [Amazon Web Services 2026a].

Este trabalho tem como objetivo implantar e avaliar uma aplicação web utilizando HPA em dois ambientes Kubernetes: um cluster local executado com

Minikube é um cluster gerenciado na AWS por meio do Amazon EKS. A mesma estratégia de autoescalamento foi submetida a testes de carga nos dois ambientes, permitindo comparar o comportamento do HPA, o consumo de recursos, o tempo de reação ao aumento e à redução da carga, a facilidade de gerenciamento e os custos associados.

2. Autoescalamento Horizontal no Kubernetes

O Kubernetes permite ajustar horizontalmente uma aplicação modificando o número de Pods que executam determinada carga de trabalho. Esse ajuste pode ser realizado automaticamente pelo *Horizontal Pod Autoscaler* (HPA), que monitora métricas associadas aos Pods e modifica o número de réplicas de recursos como Deployments de acordo com uma política previamente definida [Kubernetes 2026b].

2.1. Funcionamento do HPA

O HPA executa periodicamente um ciclo de controle no qual consulta as métricas dos Pods associados ao recurso monitorado. Para métricas de utilização de recursos, como CPU, o valor observado é relacionado aos recursos solicitados (*requests*) pelos containers. A partir da comparação entre a utilização atual e o valor alvo configurado, o controlador determina se o número de réplicas deve ser aumentado, reduzido ou mantido [Kubernetes 2026b].

Para que o escalamento baseado em CPU funcione, é necessário disponibilizar essas métricas ao Kubernetes. Uma forma de realizar essa função é por meio do Metrics Server, que coleta métricas de utilização dos nodes e Pods e as disponibiliza através da API de métricas do Kubernetes. No Amazon EKS, a AWS também indica a instalação de uma fonte de métricas, como o Metrics Server, para utilização do HPA [Amazon Web Services 2026a].

2.2. Política de escalamento

A configuração do HPA estabelece parâmetros como o número mínimo e máximo de réplicas e a utilização de recursos desejada. Quando a carga aumenta e a métrica observada supera o valor alvo, novas réplicas podem ser criadas. Com a redução da demanda, o número de réplicas pode posteriormente ser reduzido, respeitando os limites e mecanismos de estabilização definidos pelo Kubernetes [Kubernetes 2026b].

O HPA realiza o escalamento dos Pods da aplicação e não, por si só, dos nodes que compõem o cluster. Dessa forma, o aumento do número de réplicas continua limitado pela capacidade computacional disponível no cluster, aspecto particularmente relevante ao comparar um ambiente local com uma infraestrutura gerenciada em nuvem.

3. Arquitetura das Soluções

Para permitir a comparação do comportamento do HPA em diferentes ambientes, foram definidas duas implantações Kubernetes com a mesma aplicação web e política de autoescalamento. A primeira utiliza um cluster local executado com Minikube, enquanto a segunda utiliza um cluster gerenciado na AWS por meio do Amazon EKS.

3.1. Ambiente local com Minikube

Na implantação local, o Minikube é utilizado para disponibilizar um cluster Kubernetes em uma única máquina. A aplicação web é executada por meio de um Deployment e

disponibilizada por um Service. O Metrics Server fornece as métricas de utilização necessárias ao HPA, que controla automaticamente o número de réplicas da aplicação dentro dos limites configurados.

Essa arquitetura permite realizar os experimentos de autoescalamento em um ambiente local e controlado, sem utilização de infraestrutura externa.

3.2. Ambiente em nuvem com Amazon EKS

Na implantação em nuvem, o plano de controle Kubernetes é fornecido pelo Amazon EKS, enquanto os Pods da aplicação são executados nos nodes associados ao cluster. Assim como no ambiente local, são utilizados Deployment, Service, Metrics Server e HPA [Amazon Web Services 2026a].

O uso da mesma aplicação e de parâmetros equivalentes de autoescalamento nos dois ambientes busca reduzir diferenças decorrentes da configuração do experimento e permitir uma comparação direta entre a execução local e a execução em infraestrutura de nuvem.

4. Metodologia Experimental

A avaliação foi realizada por meio da implantação da mesma aplicação web nos ambientes Minikube e Amazon EKS. Sempre que possível, foram utilizados os mesmos manifestos Kubernetes, mantendo equivalentes a configuração do Deployment, os recursos solicitados pelos containers (*requests*) e os parâmetros do HPA.

4.1. Configuração do autoescalamento

O HPA foi configurado utilizando a utilização de CPU como métrica de referência, com utilização alvo de **50%**, mínimo de **uma réplica** e máximo de **dez réplicas**. No Deployment, cada container foi configurado com solicitação (*request*) de **200 millicores (200m)** e limite de **500 millicores (500m)** de CPU. Esses parâmetros foram mantidos nos ambientes Minikube e Amazon EKS, permitindo submeter as duas implantações à mesma política de autoescalamento. O Metrics Server foi utilizado para disponibilizar ao Kubernetes as métricas necessárias ao processo de decisão do HPA [Kubernetes 2026b].

Antes de cada teste foi verificado o estado inicial da aplicação, incluindo número de Pods, utilização de CPU e estado do HPA.

4.2. Teste de carga

Após a estabilização do ambiente, foi gerada uma carga contínua de requisições sobre a aplicação web com o objetivo de elevar sua utilização de CPU acima do valor alvo configurado. A carga foi gerada a partir de um Pod auxiliar executando BusyBox, denominado load-generator, que realizou continuamente requisições HTTP ao Service web-hpa durante o período de teste. Durante o experimento foram monitoradas as métricas apresentadas pelo HPA e a quantidade de réplicas do Deployment. Após a interrupção da carga, o monitoramento foi mantido para observar a redução da utilização de recursos e o posterior retorno do número de réplicas ao estado de menor demanda.

4.3. Critérios de avaliação

Os experimentos foram avaliados considerando o número de réplicas antes, durante e após a aplicação da carga, a utilização de CPU observada e os tempos de reação para aumento e redução das réplicas. Também foram considerados aspectos relacionados à configuração e ao gerenciamento de cada ambiente e, no caso do Amazon EKS, os

custos aproximados dos recursos utilizados.

Os resultados obtidos nas duas implantações foram posteriormente comparados buscando identificar diferenças no comportamento do autoescalamento e nas características operacionais dos ambientes local e em nuvem.

5. Resultados

5.1. Ambiente local com Minikube

No ambiente local, o experimento foi iniciado com uma única réplica da aplicação e utilização de CPU próxima de 0%, enquanto o HPA foi configurado com utilização alvo de 50%, mínimo de uma e máximo de dez réplicas (EV01).

Após o início da geração de carga, a utilização de CPU atingiu inicialmente 159%, provocando o aumento do número de réplicas. O primeiro escalamento observado elevou o Deployment de uma para quatro réplicas (EV02). Como a utilização permaneceu acima do valor alvo, o HPA realizou um novo escalamento, atingindo oito réplicas. Com essa configuração, a utilização média observada se estabilizou próxima do objetivo definido, com valores de aproximadamente 52% a 54% (EV03).

Após a remoção da carga, a utilização de CPU caiu rapidamente até aproximadamente 0%. A redução do número de réplicas, entretanto, não ocorreu imediatamente. O HPA manteve temporariamente as oito réplicas antes de realizar o *scale-down*, seguindo a sequência de oito para três e, posteriormente, para uma réplica. O retorno à configuração mínima ocorreu aproximadamente cinco a seis minutos após a retirada da carga (EV04).

5.2. Ambiente em nuvem com Amazon EKS

No Amazon EKS, o experimento foi executado em um cluster Kubernetes 1.36 com um único *worker node* EC2 `t3.small`. Foram mantidos os mesmos parâmetros utilizados no Minikube: uma réplica inicial, limite máximo de dez réplicas e utilização alvo de CPU de 50%. Antes da aplicação da carga, o HPA apresentava utilização próxima de 0% e uma única réplica em execução (EV05).

Com a geração de carga, a utilização de CPU aumentou significativamente e o HPA realizou sucessivos escalamentos. Durante o experimento foram observadas as configurações de uma, quatro, cinco e oito réplicas (EV06). Com oito réplicas, a utilização permaneceu predominantemente acima do alvo de 50%, oscilando aproximadamente entre 58% e 75% durante o período observado, sem que um novo aumento do número de réplicas fosse realizado (EV07).

A retirada da carga reduziu a utilização de CPU para aproximadamente 0%. Assim como no ambiente local, a redução das réplicas não ocorreu imediatamente. Após o período de estabilização do HPA, o número de réplicas foi reduzido de oito para três e posteriormente para uma, retornando à configuração inicial aproximadamente cinco minutos após a interrupção da carga (EV08).

6. Análise Comparativa

A Tabela 1 sintetiza os principais resultados obtidos nos experimentos realizados nos ambientes Minikube e Amazon EKS.

Tabela 1. Comparação dos resultados obtidos nos dois ambientes.

<i>Parâmetro</i>	<i>Minikube</i>	<i>Amazon EKS</i>
HPA alvo CPU	50%	50%
Réplicas mín./máx.	1 / 10	1 / 10
Sequência de scale-up	1 → 4 → 8	1 → 4 → 5 → 8
CPU com 8 réplicas	~52–54%	~58–75%
Scale-down observado	8 → 3 → 1	8 → 3 → 1
Retorno ao mínimo	~5–6 min	~5 min
Infraestrutura	Local	EKS + 1 t3.small
Custo direto de nuvem	—	~US\$ 0,128/h

6.1. Comportamento do autoescalamento

Os dois ambientes apresentaram comportamento semelhante diante da variação de carga. Partindo de uma única réplica, o aumento da utilização de CPU provocou rapidamente o *scale-up* da aplicação. No Minikube, foram observadas as configurações de uma, quatro e oito réplicas, enquanto no Amazon EKS foram observadas uma, quatro, cinco e oito réplicas.

Nos dois casos, a aplicação atingiu oito réplicas durante o período de carga. Entretanto, o comportamento da métrica apresentou diferenças. No Minikube, a utilização média de CPU se aproximou do alvo configurado de 50%, estabilizando-se em aproximadamente 52% a 54%. No EKS, a utilização permaneceu predominantemente entre 58% e 75% mesmo com oito réplicas. A inspeção do HPA indicou que o mecanismo permanecia ativo, com métricas válidas e sem limitação pelo número máximo de réplicas, mantendo oito como quantidade desejada durante o período observado.

Após a interrupção da carga, a utilização de CPU caiu rapidamente nos dois ambientes, enquanto a redução das réplicas ocorreu de maneira mais conservadora. Tanto no Minikube quanto no EKS foi observada a sequência de redução de oito para três e posteriormente para uma réplica, com retorno à configuração mínima após aproximadamente cinco minutos. Esse comportamento evidencia a diferença entre a reação ao aumento da demanda e a redução posterior da capacidade provisionada.

6.2. Consumo de recursos e infraestrutura

No Minikube, todos os componentes do Kubernetes e os Pods da aplicação compartilharam os recursos da máquina local. O ambiente utilizou um único node, sendo suficiente para executar as oito réplicas atingidas durante o experimento.

No Amazon EKS, o plano de controle foi fornecido como serviço gerenciado pela AWS, enquanto os Pods foram executados em um único *worker node* EC2 t3.small. Dessa forma, o HPA realizou exclusivamente o escalamento horizontal dos Pods, sem alterar a capacidade computacional do cluster. Esse aspecto evidencia que o escalamento de Pods e o escalamento da infraestrutura são mecanismos distintos: caso a capacidade do único node fosse insuficiente para receber novas réplicas, seria necessário também utilizar um mecanismo de escalamento dos nodes.

6.3. Custos

O Minikube não apresentou custo direto de infraestrutura de nuvem, utilizando recursos computacionais já disponíveis na máquina local. No Amazon EKS, por outro lado, a execução envolveu o plano de controle gerenciado, uma instância EC2 t3.small On-

Demand, um volume EBS gp3 de 20 GiB e um endereço IPv4 público.

A partir das tarifas públicas da AWS, o ambiente utilizado representa um custo estimado de aproximadamente **US\$ 0,128 por hora**, equivalente a aproximadamente **US\$ 94 por mês** considerando operação contínua durante 730 horas [Amazon Web Services 2026b, 2026c, 2026d, 2026e]. O plano de controle do EKS constitui a maior parcela dessa estimativa. Não foram utilizados Load Balancer ou NAT Gateway no experimento.

A consulta ao AWS Cost Explorer realizada imediatamente após os testes ainda não apresentava os serviços utilizados no experimento, indicando que a contabilização não havia sido consolidada naquele momento. Por esse motivo, a comparação utiliza a estimativa calculada a partir dos recursos efetivamente provisionados e das tarifas publicadas pelo provedor.

6.4. Implantação e gerenciamento

O Minikube apresentou menor complexidade para a realização do experimento. O cluster local já disponibilizava os componentes básicos do Kubernetes e o Metrics Server pôde ser habilitado diretamente como um *addon*. A implantação da aplicação, do Service e do HPA foi então realizada exclusivamente por meio dos objetos Kubernetes.

No EKS, embora o plano de controle seja gerenciado pela AWS, foi necessário provisionar previamente recursos adicionais, incluindo VPC e sub-redes, funções e políticas IAM, cluster EKS e grupo de nodes EC2. Também foi necessária a instalação do Metrics Server antes da utilização do HPA. Além disso, os recursos provisionados precisam ser removidos após o experimento para evitar custos desnecessários.

Por outro lado, o EKS transfere ao provedor o gerenciamento do plano de controle Kubernetes e permite integrar o cluster aos demais serviços e mecanismos de infraestrutura da AWS. Assim, a maior complexidade de provisionamento observada no experimento está associada às possibilidades de operação em uma infraestrutura de nuvem gerenciada, enquanto o Minikube se mostrou mais simples e adequado à experimentação local.

7. Conclusões

Este trabalho avaliou o funcionamento do Horizontal Pod Autoscaler em duas implantações Kubernetes: um cluster local utilizando Minikube e um cluster gerenciado na AWS utilizando Amazon EKS. Em ambos os ambientes, o HPA respondeu ao aumento da utilização de CPU por meio da criação automática de novas réplicas e, após a retirada da carga, retornou à configuração mínima estabelecida.

Os experimentos mostraram comportamento semelhante nos dois ambientes, embora tenham sido observadas diferenças durante o *scale-up*. No Minikube, o número de réplicas evoluiu de uma para quatro e posteriormente para oito, com a utilização de CPU estabilizando próxima ao alvo de 50%. No EKS, foram observadas as configurações de uma, quatro, cinco e oito réplicas, permanecendo a utilização de CPU um pouco acima do valor alvo durante parte significativa do experimento. Em ambos os casos, o *scale-down* ocorreu de maneira mais conservadora, com redução de oito para três e posteriormente para uma réplica, aproximadamente cinco minutos após a retirada da carga.

A comparação também evidenciou diferenças importantes de infraestrutura. O Minikube proporcionou uma implantação mais simples e sem custos adicionais de serviços de nuvem, sendo adequado para desenvolvimento e experimentação. O EKS exigiu o provisionamento de recursos adicionais de rede, IAM e computação, além de apresentar custo estimado de aproximadamente US\$ 94 mensais para a configuração

mínima utilizada. Em contrapartida, oferece um plano de controle Kubernetes gerenciado e integração com a infraestrutura da AWS.

Por fim, os resultados demonstram que o HPA permite adaptar automaticamente a quantidade de réplicas à demanda da aplicação, mas também evidenciam que o escalamento de Pods não implica aumento da capacidade computacional do cluster. Em cenários de produção sujeitos a variações maiores de carga, o HPA deve ser combinado com mecanismos capazes de ajustar também a quantidade ou capacidade dos *worker nodes*, permitindo que novas réplicas encontrem recursos computacionais disponíveis.

Declaração de Uso de Inteligência Artificial

Durante a elaboração deste trabalho foi utilizada a ferramenta ChatGPT, da OpenAI, como apoio à organização do conteúdo e revisão textual de trechos do artigo e da documentação. A implantação dos ambientes, execução dos experimentos, coleta das evidências e análise dos resultados foram realizadas pelo autor, que também realizou a revisão e validação do conteúdo final.

Referências

Amazon Web Services (2026a). “Scale pod deployments with Horizontal Pod Autoscaler”, *Amazon EKS User Guide*.

<https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>.

Amazon Web Services (2026b). “Amazon EKS Pricing”, *Amazon Web Services*.

<https://aws.amazon.com/eks/pricing/>.

Amazon Web Services (2026c). “Amazon EC2 On-Demand Pricing”, *Amazon Web Services*.

<https://aws.amazon.com/ec2/pricing/on-demand/>.

Amazon Web Services (2026d). “Amazon EBS Pricing”, *Amazon Web Services*.

<https://aws.amazon.com/ebs/pricing/>.

Amazon Web Services (2026e). “Amazon VPC Pricing”, *Amazon Web Services*.

<https://aws.amazon.com/vpc/pricing/>.

Kubernetes (2026a). “Deployments”, *Kubernetes Documentation*.

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>.

Kubernetes (2026b). “Horizontal Pod Autoscaling”, *Kubernetes Documentation*.

<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>.

Anexo A – Relação das evidências experimentais

As evidências de execução utilizadas para validação dos experimentos estão disponibilizadas na pasta evidencias do arquivo compactado entregue juntamente com este artigo. Os arquivos estão identificados conforme a relação a seguir.

Tabela 2. Descrição dos arquivos de evidências.

<i>Identificação</i>	<i>Arquivo</i>	<i>Descrição</i>
EV01	ev01_local_estado_inicial.png	Estado inicial do experimento no Minikube, com uma réplica e HPA em 0%/50%.
EV02	ev02_local_hpa_scaleup.png	Aumento da utilização de CPU e início do escalamento horizontal no Minikube.
EV03	ev03_local_hpa_estabilizado.png	Estabilização do ambiente local em oito réplicas durante a aplicação da carga.
EV04	ev04_local_hpa_final.png	Redução das réplicas após a retirada da carga e retorno do Minikube ao estado inicial.
EV05	ev05_eks_estado_inicial.png	Estado inicial do experimento no Amazon EKS, com uma réplica e HPA em 0%/50%.
EV06	ev06_eks_scaleup.png	Escalamento horizontal observado no Amazon EKS durante a geração de carga.
EV07	ev07_eks_estabilizacao.png	Comportamento do EKS com oito réplicas e utilização de CPU acima do alvo configurado.
EV08	ev08_eks_scaledown.png	Redução das réplicas no EKS após a retirada da carga, com retorno à configuração mínima.